

PANDUAN OOP (OBJECT-ORIENTED PROGRAMMING)

Aplikasi Kasir Baju

Dokumen ini menjelaskan secara detail bagian-bagian mana saja di dalam proyek Aplikasi Kasir Baju yang mengandung unsur OOP (Object-Oriented Programming).
Dibuat agar dapat belajar, memahami, dan menjawab pertanyaan dosen dengan percaya diri.

Daftar Isi

1. Apa Itu OOP?
2. 4 Pilar Utama OOP
3. Pilar 1 - Inheritance (Pewarisan)
4. Pilar 2 - Encapsulation (Pengkapsulan)
5. Pilar 3 - Polymorphism (Banyak Bentuk)
6. Pilar 4 - Abstraction (Abstraksi)
7. Konsep OOP Tambahan
8. Ringkasan Tabel per File
9. Contoh Pertanyaan Dosen dan Jawaban

1. Apa Itu OOP?

OOP (Object-Oriented Programming) adalah paradigma pemrograman yang mengorganisasi kode ke dalam objek-objek yang terdiri dari data (properties/atribut) dan fungsi (methods/perilaku).

Dalam proyek Laravel ini, setiap Model, Controller, dan Middleware adalah class (kelas), yaitu cetakan atau blueprint untuk membuat objek.

Contoh sederhana:

Class Produk = Cetakan/Blueprint
Objek = Data produk "Kemeja Flanel" yang dibuat dari cetakan tersebut

2. 4 Pilar Utama OOP

No	Pilar	Bahasa Indonesia	Penjelasan Singkat
1	Inheritance	Pewarisan	Class anak mewarisi sifat dari class induk
2	Encapsulation	Pengkapsulan	Menyembunyikan data internal, hanya bisa diakses lewat cara tertentu
3	Polymorphism	Banyak Bentuk	Satu method/fungsi bisa berperilaku berbeda di class yang berbeda
4	Abstraction	Abstraksi	Menyembunyikan detail kompleks, hanya menampilkan fitur yang diperlukan

3. Pilar 1 - Inheritance (Pewarisan)

Definisi

Inheritance adalah kemampuan suatu class (child/anak) untuk mewarisi semua properties dan methods dari class lain (parent/induk). Keyword-nya di PHP adalah extends.

Dimana Ditemukan di Proyek

A. Semua Model mewarisi Model atau Authenticatable

File: app/Models/Kategori.php

```
class Kategori extends Model // Inheritance
{
    // Kategori mewarisi semua kemampuan dari class Model Laravel
    // seperti: create(), update(), delete(), find(), all(), dsb.
}
```

File: app/Models/Produk.php

```
class Produk extends Model // Inheritance
{
    // Produk juga mewarisi semua kemampuan dari class Model
}
```

File: app/Models/User.php

```
class User extends Authenticatable // Inheritance
{
    // User mewarisi dari Authenticatable (turunan dari Model juga)
    // Sehingga User mendapat kemampuan login, logout, autentikasi, dll.
}
```

Semua 8 Model di proyek ini menggunakan Inheritance: User, Kategori, Produk, VarianProduk, Warna, Ukuran, Penjualan, DetailPenjualan.

B. Semua Controller mewarisi Controller

File: app/Http/Controllers/AuthController.php

```
class AuthController extends Controller // Inheritance
{
    // AuthController mewarisi dari base Controller
}
```

File: app/Http/Controllers/ProdukController.php

```
class ProdukController extends Controller // Inheritance
{
    // ProdukController mewarisi dari base Controller
}
```

Semua 10 Controller di proyek ini menggunakan Inheritance: AuthController, DashboardController, KategoriController, WarnaController, UkuranController,

ProdukController, StokController, UserController, PenjualanController, LaporanController.

C. Migration menggunakan Anonymous Class yang mewarisi Migration

File: database/migrations/2026_07_03_000004_create_produk_table.php

```
return new class extends Migration // Inheritance (Anonymous Class)
{
    public function up(): void { ... }
    public function down(): void { ... }
};
```

Semua 10 file migration menggunakan pola ini.

D. Base Controller sebagai Abstract Class

File: app/Http/Controllers/Controller.php

```
abstract class Controller // Abstract Class (bagian dari Inheritance dan Abstraction)
{
    //
}
```

Penjelasan untuk Dosen

"Di proyek saya, semua Model mewarisi class Model dari Laravel menggunakan keyword extends. Ini artinya setiap Model seperti Produk, Kategori, Penjualan secara otomatis mendapatkan method-method bawaan seperti create(), update(), delete(), find(), dan all() tanpa harus menulis ulang method-method tersebut. Begitu juga semua Controller yang mewarisi base Controller."

4. Pilar 2 - Encapsulation (Peng kapsulan)

Definisi

Encapsulation adalah konsep membungkus atau menyembunyikan data internal suatu class dan hanya menyediakan akses melalui method tertentu. Menggunakan access modifier: public, protected, private.

Dimana Ditemukan di Proyek

A. Property protected pada semua Model

File: app/Models/Produk.php

```
class Produk extends Model
{
    protected $table = 'produk';      // protected, tidak bisa diakses langsung dari luar class
    protected $primaryKey = 'id_produk'; // protected
    protected $fillable = [           // protected, mengontrol kolom mana yang boleh diisi massal
        'id_kategori',
        'nama_produk',
        'deskripsi',
        'foto_produk',
    ];
}
```

Penjelasan protected:

- protected \$table -> Hanya class ini dan turunannya yang bisa mengakses nama tabel.
- protected \$fillable -> Hanya kolom yang disebutkan yang boleh di-insert/update secara massal. Ini melindungi dari Mass Assignment Attack (keamanan data).
- Dari luar class, tidak bisa langsung menulis \$produk->fillable = [...].

B. Property \$hidden pada Model User

File: app/Models/User.php

```
protected $hidden = [
    'password',      // Disembunyikan saat dikonversi ke JSON/array
    'remember_token', // Disembunyikan saat dikonversi ke JSON/array
];
```

Ketika data User diubah ke format JSON (misalnya untuk API), kolom password dan remember_token tidak akan ditampilkan. Ini adalah Encapsulation untuk melindungi data sensitif.

C. Validasi Input di Controller (Encapsulation Logic)

File: app/Http/Controllers/KategoriController.php

```
public function store(Request $request): RedirectResponse
{
    // Validasi input, logika proteksi data dienkapsulasi di dalam method ini
}
```

```

$request->validate([
    'nama_kategori' => 'required|string|max:255|unique:kategori,nama_kategori',
]);

Kategori::create([
    'nama_kategori' => $request->nama_kategori,
]);
// ...
}

```

Logika validasi tersembunyi di dalam method `store()`. Pemanggil method (yaitu route) tidak perlu tahu bagaimana validasi dilakukan, cukup memanggil saja.

D. Password Hashing di UserController

File: `app/Http/Controllers/UserController.php`

```

User::create([
    'nama' => $request->nama,
    'username' => $request->username,
    'role' => $request->role,
    'password' => Hash::make($request->password), // Encapsulation, password asli
    disembunyikan
]);

```

Password tidak pernah disimpan dalam bentuk aslinya. Nilai asli dibungkus menjadi hash yang tidak bisa dibaca, ini adalah Encapsulation terhadap data sensitif.

E. Middleware mengenkapsulasi logika otorisasi

File: `app/Http/Middleware/AdminMiddleware.php`

```

public function handle(Request $request, Closure $next): Response
{
    if (Auth::check()) {
        if (Auth::user()->role === 'Admin') {
            return $next($request); // Lanjutkan ke controller
        }
        return redirect('/dashboard')->with('error', 'Akses ditolak!');
    }
    return redirect('/login');
}

```

Seluruh logika "apakah user ini Admin?" dienkapsulasi di dalam class AdminMiddleware. Controller tidak perlu mengecek role secara manual, cukup pasang middleware saja.

Penjelasan untuk Dosen

"Encapsulation diterapkan melalui access modifier protected pada properties Model seperti \$table, \$primaryKey, \$fillable, dan \$hidden. Property \$fillable secara khusus melindungi dari serangan Mass Assignment. Selain itu, password user di-hash sebelum disimpan sehingga nilai aslinya tersembunyi, dan logika otorisasi di-enkapsulasi di dalam Middleware."

5. Pilar 3 - Polymorphism (Banyak Bentuk)

Definisi

Polymorphism adalah kemampuan method/fungsi yang sama namanya tetapi bisa berperilaku berbeda tergantung pada class atau konteks pemanggilannya.

Dimana Ditemukan di Proyek

A. Method casts() di-override pada Model User

File: app/Models/User.php

```
protected function casts(): array // Override method bawaan Model
{
    return [
        'password' => 'hashed',
    ];
}
```

Method casts() sudah ada di class induk Model, tetapi class User mendefinisikan ulang (override) method tersebut untuk kebutuhan spesifik. Ini adalah Method Overriding, salah satu bentuk Polymorphism.

B. Method CRUD yang sama namanya tapi berbeda implementasi

Perhatikan bahwa banyak Controller punya method dengan nama yang sama (index(), store(), update(), destroy()) tetapi implementasinya berbeda:

```
// KategoriController
public function store(Request $request)
{
    Kategori::create(['nama_kategori' => $request->nama_kategori]);
}

// ProdukController
public function store(Request $request)
{
    // Lebih kompleks: ada upload foto, validasi tambahan
    if ($request->hasFile('foto_produk')) {
        $path = $request->file('foto_produk')->store('produk', 'public');
    }
    Produk::create([...]);
}

// PenjualanController
public function store(Request $request)
{
    // Paling kompleks: DB Transaction, validasi stok, potong stok, generate nomor transaksi
    DB::transaction(function () use ($request) { ... });
}
```

Ketiga method bernama store() tapi perilakunya sangat berbeda, ini mewakili prinsip Polymorphism.

C. Relasi Eloquent - Method sama, hasil berbeda

```
// Di Model Kategori: hasMany, mengembalikan BANYAK Produk
public function produk(): HasMany
{
    return $this->hasMany(Produk::class, 'id_kategori', 'id_kategori');
}

// Di Model Produk: belongsTo, mengembalikan SATU Kategori
public function kategori(): BelongsTo
{
    return $this->belongsTo(Kategori::class, 'id_kategori', 'id_kategori');
}
```

Method hasMany() dan belongsTo() adalah method bawaan Model yang berperilaku berbeda tergantung cara pemanggilannya, ini juga bentuk Polymorphism.

D. Redirect berdasarkan Role (Polymorphic Behavior)

File: app/Http/Controllers/AuthController.php

```
if ($user->role === 'Admin') {  
    return redirect('/admin/dashboard'); // Perilaku A  
} elseif ($user->role === 'Kasir') {  
    return redirect('/kasir/dashboard'); // Perilaku B  
}
```

Satu method login() menghasilkan perilaku yang berbeda (redirect ke halaman berbeda) berdasarkan tipe/role user. Ini dikenal sebagai Polymorphic Behavior, konsep Polymorphism yang diterapkan lewat conditional logic.

Penjelasan untuk Dosen

"Polymorphism diterapkan melalui Method Overriding, contohnya method casts() di Model User yang meng-override method bawaan dari class induk Model. Selain itu, method store() yang ada di KategoriController, ProdukController, dan PenjualanController memiliki nama sama tapi implementasi berbeda sesuai kebutuhan masing-masing entitas."

6. Pilar 4 - Abstraction (Abstraksi)

Definisi

Abstraction adalah proses menyembunyikan detail implementasi yang rumit dan hanya menampilkan fitur/fungsi yang diperlukan pengguna. Pengguna cukup tahu "apa yang dilakukan", tanpa perlu tahu "bagaimana cara kerjanya".

Dimana Ditemukan di Proyek

A. Abstract Class Controller

File: app/Http/Controllers/Controller.php

```
abstract class Controller // Keyword "abstract"  
{  
    //  
}
```

Keyword abstract berarti class ini tidak bisa diinstansiasi langsung, hanya bisa digunakan sebagai class induk untuk diwariskan. Ini memaksa semua controller turunannya untuk mengimplementasikan method-method sendiri.

B. Eloquent ORM sebagai Abstraksi Database

File: app/Http/Controllers/KategoriController.php

```
// TANPA OOP (Query SQL Manual):  
// $result = mysqli_query($conn, "SELECT * FROM kategori");  
  
// DENGAN OOP (Eloquent ORM, Abstraction):  
$kategori = Kategori::all(); // Cukup panggil ini saja  
  
// TANPA OOP (Query SQL Manual):  
// $sql = "INSERT INTO kategori (nama_kategori) VALUES ('Pakaian Pria')";  
  
// DENGAN OOP (Eloquent ORM, Abstraction):  
Kategori::create(['nama_kategori' => 'Pakaian Pria']); // Jauh lebih simpel
```

Tidak perlu menulis query SQL mentah. Eloquent ORM mengabstraksi seluruh proses query database menjadi method-method sederhana seperti all(), create(), update(), delete(), find(), where(), dll.

C. Auth Facade sebagai Abstraksi Sistem Autentikasi

File: app/Http/Controllers/AuthController.php

```
Auth::attempt($credentials); // Abstraction  
Auth::user(); // Abstraction  
Auth::check(); // Abstraction  
Auth::logout(); // Abstraction
```

Di balik layar, Auth::attempt() melakukan:

- Query database untuk mencari username
- Mengambil hash password dari database
- Membandingkan password yang diinput dengan hash
- Membuat session jika cocok

Tapi tidak perlu tahu semua langkah itu, cukup panggil Auth::attempt() saja.

D. DB Transaction sebagai Abstraksi Integritas Data

File: app/Http/Controllers/PenjualanController.php

```
$result = DB::transaction(function () use ($request) {  
    // Semua operasi di sini DIJAMIN atomik:  
    // - Jika ada error, SEMUA perubahan dibatalkan (rollback)  
    // - Jika sukses, SEMUA perubahan disimpan (commit)  
  
    $penjualan = Penjualan::create([...]);  
    DetailPenjualan::create([...]);  
    $varian->stok = $varian->stok - $data['qty'];  
    $varian->save();  
});
```

DB::transaction() mengabstraksi mekanisme BEGIN TRANSACTION, COMMIT, dan ROLLBACK pada database. Tidak perlu menulis kode SQL transaction secara manual.

E. Hash Facade sebagai Abstraksi Kriptografi

File: app/Http/Controllers/UserController.php

```
'password' => Hash::make($request->password),
```

Hash::make() mengabstraksi algoritma hashing bcrypt yang kompleks. Tidak perlu memahami algoritma kriptografi, cukup panggil satu method saja.

Penjelasan untuk Dosen

"Abstraction diterapkan melalui penggunaan abstract class Controller sebagai base class, Eloquent ORM yang mengabstraksi query SQL menjadi method sederhana seperti all() dan create(), Auth Facade yang menyembunyikan kompleksitas proses autentikasi, dan DB::transaction() yang mengabstraksi mekanisme commit/rollback database."

7. Konsep OOP Tambahan

A. Class dan Object (Kelas dan Objek)

```
// Class = Cetakan/Blueprint  
class Produk extends Model { ... }
```

```
// Object = Instance dari class (dibuat saat runtime)
$produk = Produk::find(1);    // $produk adalah OBJEK dari class Produk
$produk = new Produk();       // Membuat objek baru dari class Produk
$kategori = Kategori::all();   // Mengembalikan KUMPULAN objek Kategori
```

B. Association (Asosiasi/Relasi antar Objek)

Asosiasi menggambarkan hubungan antar objek. Di proyek ini, ini diimplementasikan lewat Eloquent Relationships:

Tipe Relasi	Contoh di Proyek	File
One to Many (1:N)	Satu Kategori memiliki banyak Produk	Kategori.php -> produk()
One to Many (1:N)	Satu Produk memiliki banyak VarianProduk	Produk.php -> varianProduk()
One to Many (1:N)	Satu User memiliki banyak Penjualan	User.php -> penjualan()
One to Many (1:N)	Satu Penjualan memiliki banyak DetailPenjualan	Penjualan.php -> detailPenjualan()
One to Many (1:N)	Satu Warna memiliki banyak VarianProduk	Warna.php -> varianProduk()
One to Many (1:N)	Satu Ukuran memiliki banyak VarianProduk	Ukuran.php -> varianProduk()
Many to One (N:1)	Banyak Produk dimiliki satu Kategori	Produk.php -> kategori()
Many to One (N:1)	Banyak VarianProduk dimiliki satu Produk	VarianProduk.php -> produk()
Many to One (N:1)	Banyak VarianProduk dimiliki satu Warna	VarianProduk.php -> warna()
Many to One (N:1)	Banyak VarianProduk dimiliki satu Ukuran	VarianProduk.php -> ukuran()
Many to One (N:1)	Banyak Penjualan dimiliki satu User	Penjualan.php -> user()
Many to One (N:1)	Banyak DetailPenjualan dimiliki satu Penjualan	DetailPenjualan.php -> penjualan()
Many to One (N:1)	Banyak DetailPenjualan dimiliki satu VarianProduk	DetailPenjualan.php -> varianProduk()

C. Trait (Use)

File: app/Models/User.php

```
class User extends Authenticatable
{
```

```
use HasFactory, Notifiable; // Trait
}
```

Trait adalah mekanisme PHP untuk berbagi method ke banyak class tanpa menggunakan Inheritance. Ini solusi karena PHP tidak mendukung Multiple Inheritance (mewarisi dari banyak class sekaligus). Trait HasFactory memberikan kemampuan factory pattern, sedangkan Notifiable memberikan kemampuan menerima notifikasi.

D. Namespace

```
namespace App\Models;           // Namespace untuk Model
namespace App\Http\Controllers; // Namespace untuk Controller
namespace App\Http\Middleware;   // Namespace untuk Middleware
```

Namespace mengorganisasi class-class ke dalam "folder logis" agar tidak terjadi konflik nama (misalnya ada class User di library lain, tidak akan bentrok dengan App\Models\User milik kita).

E. Type Hinting dan Return Type

```
public function store(Request $request): RedirectResponse // Type Hinting
{
    // $request bertipe Request (Type Hinting parameter)
    // Method ini HARUS mengembalikan RedirectResponse (Return Type Declaration)
}

public function penjualan(): BelongsTo // Return Type
{
    return $this->belongsTo(Penjualan::class, ...);
}
```

Type Hinting memastikan parameter dan return value sesuai tipe yang diharapkan, ini meningkatkan keamanan tipe data (Type Safety) yang merupakan bagian dari prinsip OOP.

F. Method Chaining (Fluent Interface)

```
$stok = VarianProduk::with(['produk', 'warna', 'ukuran'])
->where('stok', '<=', 5)
->orderBy('stok', 'asc')
```

```
->limit(5)
->get();
```

Setiap method mengembalikan objek dirinya sendiri sehingga bisa dipanggil secara berantai. Ini disebut Fluent Interface atau Method Chaining, pola desain OOP yang membuat kode lebih ringkas dan mudah dibaca.

G. Closure / Anonymous Function

```
DB::transaction(function () use ($request) { // Anonymous Function / Closure
    // ...
});
```

Closure adalah fungsi tanpa nama yang bisa disimpan dalam variabel atau dikirim sebagai parameter. Di PHP, closure digunakan bersama OOP untuk pattern seperti Callback dan Strategy.

8. Ringkasan Tabel per File

Models (app/Models/)

File	Inheritance	Encapsulation	Polymorphism	Abstraction	Konsep Lain
User.php	extends Authenticatable	protected \$fillable, \$hidden	Override casts()	Auth system	Trait (use HasFactory, Notifiable)
Kategori.php	extends Model	protected \$table, \$fillable	-	Eloquent ORM	Association (hasMany)
Produk.php	extends Model	protected \$table, \$fillable	-	Eloquent ORM	Association (belongsTo, hasMany)
VarianProduk.php	extends Model	protected \$table, \$fillable	-	Eloquent ORM	Association (3x belongsTo, 1x hasMany)
Penjualan.php	extends Model	protected \$table, \$fillable	-	Eloquent ORM	Association (belongsTo, hasMany)
DetailPenjualan.php	extends Model	protected \$table, \$fillable	-	Eloquent ORM	Association (2x belongsTo)

File	Inheritance	Encapsulation	Polymorphism	Abstraction	Konsep Lain
Warna.php	extends Model	protected \$table, \$fillable	-	Eloquent ORM	Association (hasMany)
Ukuran.php	extends Model	protected \$table, \$fillable	-	Eloquent ORM	Association (hasMany)

Controllers (app/Http/Controllers/)

File	Inheritance	Encapsulation	Polymorphism	Abstraction	Konsep Lain
Controller.php	-	-	-	abstract class	Abstract Class
AuthController.php	extends Controller	Validasi login	Redirect per role	Auth::attempt()	Facade
DashboardController.php	extends Controller	Logic query	-	Eloquent query	Aggregation
KategoriController.php	extends Controller	Validasi CRUD	store()	Kategori::create()	CRUD Pattern
ProdukController.php	extends Controller	Upload + Validasi	store()	Storage::disk()	File Handling
StokController.php	extends Controller	Validasi stok	store()	Eloquent query	Batch Create
UserController.php	extends Controller	Hash password	store()	Hash::make()	Security
PenjualanController.php	extends Controller	Transaction logic	store()	DB::transaction()	Exception Handling
LaporanController.php	extends Controller	CSV streaming	exportExcel() per jenis	StreamedResponse	Export Pattern
UkuranController.php	extends Controller	Validasi input	store()	Ukuran::create()	CRUD Pattern
WarnaController.php	extends Controller	Validasi input	store()	Warna::create()	CRUD Pattern

Middleware (app/Http/Middleware/)

File	Inheritance	Encapsulation	Polymorphism	Abstraction
AdminMiddleware.php	-	Logic otorisasi	-	Auth::check()
KasirMiddleware.php	-	Logic otorisasi	-	Auth::check()

Migrations (database/migrations/)

File	Inheritance	Konsep Lain
Semua 10 file migration	extends Migration	Anonymous Class, Blueprint Pattern

9. Contoh Pertanyaan Dosen dan Jawaban

Q1: Apa itu Inheritance dan dimana contohnya di proyek kamu?

"Inheritance atau pewarisan adalah kemampuan class anak mewarisi semua properties dan methods dari class induk menggunakan keyword extends. Di proyek saya, contohnya semua Model seperti Kategori, Produk, Penjualan meng-extends class Model dari Laravel, sehingga otomatis mendapat method create(), update(), delete(), find(), dan all() tanpa menulis ulang. Class User meng-extends Authenticatable yang merupakan turunan dari Model, sehingga mendapat kemampuan autentikasi tambahan. Semua Controller meng-extends base Controller yang merupakan abstract class."

Q2: Jelaskan Encapsulation di proyek kamu!

"Encapsulation adalah membungkus data internal agar tidak bisa diakses sembarangan. Di proyek saya, semua property Model seperti \$table, \$primaryKey, dan \$fillable menggunakan access modifier protected, artinya hanya class itu sendiri dan turunannya yang bisa mengakses. Property \$fillable membatasi kolom mana saja yang boleh diisi secara massal, melindungi dari serangan Mass Assignment. Property \$hidden di Model User menyembunyikan kolom password dan remember_token saat data dikonversi ke JSON. Password di-hash menggunakan Hash::make() sebelum disimpan, sehingga nilai aslinya tersembunyi."

Q3: Apa itu Polymorphism? Berikan contoh!

"Polymorphism artinya 'banyak bentuk', method yang sama bisa berperilaku berbeda. Di proyek saya, method casts() di Model User meng-override method bawaan dari class induk Model, ini disebut Method Overriding. Method store() ada

di KategoriController, ProdukController, PenjualanController, namanya sama tapi implementasinya berbeda: KategoriController hanya menyimpan nama, ProdukController menangani upload foto, PenjualanController mengelola transaction dan stok. Method login() di AuthController menghasilkan redirect berbeda tergantung role user (Admin ke admin dashboard, Kasir ke kasir dashboard)."

Q4: Dimana Abstraction diterapkan?

*"Abstraction adalah menyembunyikan detail kompleks dan hanya menampilkan fungsi yang diperlukan. Di proyek saya, base Controller dideklarasikan sebagai abstract class, tidak bisa diinstansiasi langsung. Eloquent ORM mengabstraksi query SQL: daripada menulis SELECT * FROM kategori, cukup panggil Kategori::all(). Auth::attempt() mengabstraksi seluruh proses login: query database, compare hash password, buat session, semua cukup satu baris kode. DB::transaction() mengabstraksi mekanisme BEGIN TRANSACTION, COMMIT, dan ROLLBACK."*

Q5: Apa perbedaan protected dan private di proyek kamu?

"Di proyek saya menggunakan protected pada properties Model seperti \$table, \$fillable, dan \$hidden. protected berarti property bisa diakses oleh class itu sendiri dan class turunannya, ini penting karena semua Model kita mewarisi dari class Model Laravel yang perlu mengakses property-property tersebut. Jika menggunakan private, class induk Model tidak bisa membaca property tersebut dan aplikasi akan error."

Q6: Jelaskan hubungan antar class di proyek kamu!

Jawaban: "Di proyek saya ada hubungan Association antar class yang diimplementasikan lewat Eloquent Relationships:

- Satu Kategori memiliki banyak Produk (One to Many), method hasMany()
- Satu Produk dimiliki satu Kategori (Many to One), method belongsTo()
- Satu Produk memiliki banyak VarianProduk (One to Many)
- Satu VarianProduk terhubung ke satu Warna dan satu Ukuran (Many to One)
- Satu Penjualan memiliki banyak DetailPenjualan (One to Many)

- Satu User dapat membuat banyak Penjualan (One to Many)"

Q7: Apa itu Trait dan kenapa digunakan?

"Trait adalah fitur PHP untuk berbagi method ke banyak class tanpa Inheritance. Digunakan karena PHP tidak mendukung Multiple Inheritance. Di Model User saya menggunakan use HasFactory, Notifiable: HasFactory memberikan kemampuan Factory Pattern untuk testing, sedangkan Notifiable memberikan kemampuan menerima notifikasi sistem."

Tips Menghadapi Dosen

1. Selalu sebutkan nama file dan baris kode saat menjelaskan.
2. Jelaskan mengapa konsep OOP digunakan, bukan hanya apa itu.
3. Bandingkan pendekatan OOP dengan tanpa OOP untuk menunjukkan keuntungannya.
4. Gunakan analogi sederhana: Class itu seperti cetakan kue, Object itu kuenya.

*Dokumen ini dibuat berdasarkan analisis mendalam terhadap seluruh source code proyek Aplikasi Kasir Baju.
Terakhir diperbarui: 5 Juli 2026*